Tagless Final

s Final · User registration against UserRepo/Email/Logger algebras; IO · Build with

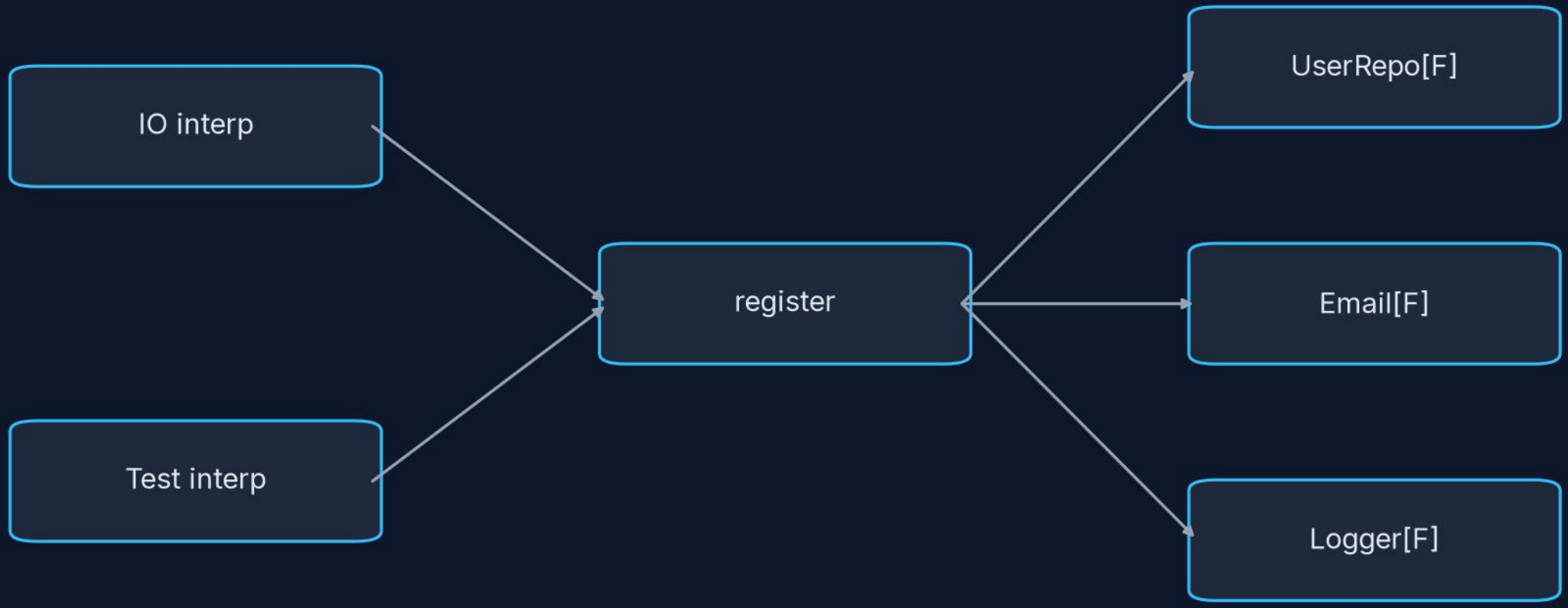
Today's Hook

- Problem: User registration against UserRepo/Email/Logger algebras; IO + pure test interpreters
- Pattern: Tagless Final — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- User registration against UserRepo/Email/Logger algebras; IO + pure test interpreters
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Tagless Final: User registration against UserRepo/Email/Logger al

Real-World Scenario

- Running example: User registration against UserRepo/Email/Logger algebras; IO + pure test interpreters
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.tagless.problem;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/** Naive: registration hard-wires Map DB + System.out email — untestable without mocks frameworks. */
public final class NaiveRegistration {
    private final Map<String, Algebras.User> db = new LinkedHashMap<>();
    private final List<String> outbox = new ArrayList<>();

    public Algebras.User register(String id, String email) {
        if (db.containsKey(email)) {
            System.out.println("dup " + email); // side effect baked in
            return db.get(email);
        }
        var u = new Algebras.User(id, email);
        db.put(email, u);
        outbox.add("welcome->" + email); // cannot swap for test assertion easily in unit style
        System.out.println("sent welcome to " + email);
        return u;
    }
```

Naive Code (2)

```
    public List<String> outbox() { return List.copyOf(outbox); }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.tagless.problem;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.function.Function;

/**
 * Tagless Final algebras for user registration.
 * Programs are written once against UserRepo/EmailSender/Logger; interpreters choose the effect.
 */
public final class Algebras {
    private Algebras() {}

    public record User(String id, String email) {}

    /** Minimal "effect" wrapper — either Identity (prod sync) or accumulating TestState. */
    public interface Kind<F, A> {
        A unsafeRun();
    }
```

Solution (2)

```
public interface Monad<F> {
    <A> Kind<F, A> pure(A a);
    <A, B> Kind<F, B> flatMap(Kind<F, A> fa, Function<A, Kind<F, B>> f);
    default <A, B> Kind<F, B> map(Kind<F, A> fa, Function<A, B> f) {
        return flatMap(fa, a -> pure(f.apply(a)));
    }
}

public interface UserRepo<F> {
    Kind<F, Optional<User>> findByEmail(String email);
    Kind<F, User> save(User user);
}

public interface EmailSender<F> {
    Kind<F, Void> send(String to, String subject, String body);
}

public interface Logger<F> {
    Kind<F, Void> info(String msg);
}
```

Solution (3)

```
/** Shared registration program — no concrete effect type. */
public static <F> Kind<F, User> register(
        Monad<F> M, UserRepo<F> users, EmailSender<F> email, Logger<F> log,
        String id, String address) {
    return M.flatMap(log.info("register " + address), ignored ->
            M.flatMap(users.findByEmail(address), existing -> {
                if (existing.isPresent()) {
                    return M.flatMap(log.info("duplicate " + address), x ->
                            M.pure(existing.get())); // return existing — caller may treat as failure
                }
                var user = new User(id, address);
                return M.flatMap(users.save(user), saved ->
                        M.flatMap(email.send(address, "Welcome", "Hi " + id), sent ->
                                M.flatMap(log.info("registered " + id), y -> M.pure(saved))));
            }));
}

// ---- Identity / "IO-style" interpreter (eager, side-effecting) ----
public enum IoWitness { W }
public record Io<A>(A value) implements Kind<IoWitness, A> {
    public A unsafeRun() { return value; }
}
```

Solution (4)

```
public static final Monad<IoWitnes
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.tagless.problem;

/** Problem: same registration program under IO-style and pure test interpreters. */
public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Tagless Final: user registration algebras ===");
        System.out.println("--- naive (hard-wired Map + System.out) ---");
        var naive = new NaiveRegistration();
        System.out.println(naive.register("u1", "a@b.c"));
        System.out.println("outbox=" + naive.outbox());

        System.out.println("--- IO-style interpreter ---");
        var repo = new Algebras.IoUserRepo();
        var mail = new Algebras.IoEmail();
        var log = new Algebras.IoLogger();
        var user = Algebras.register(Algebras.IO, repo, mail, log, "u2", "x@y.z").unsafeRun();
        System.out.println("user=" + user);
        System.out.println("emails=" + mail.sent());
        System.out.println("logs=" + log.lines());

        System.out.println("--- pure test interpreter (shared state) ---");
        var state = Algebras.TestState.empty();
        var prog = Algebras.register(
                Algebras.TEST, new Algebras.TestUserRepo(), new Algebras.TestEmail(), new Algebras.TestLogger(),
                "u3", "t@est.io");
        var saved = ((Algebras.Test<Algebras.User>) prog).runWith(state);
        System.out.println("user=" + saved);
        System.out.println("emails=" + state.emails());
        System.out.println("logs=" + state.logs());
        System.out.println("db=" + state.users().keySet());
    }
```

Runnable main (2)

```
}
```

Walkthrough Output

```
=== Tagless Final: user registration algebras ===
--- naive (hard-wired Map + System.out) ---
sent welcome to a@b.c
User[id=u1, email=a@b.c]
outbox=[welcome->a@b.c]
--- IO-style interpreter ---
user=User[id=u2, email=x@y.z]
emails=[x@y.z|Welcome|Hi u2]
logs=[register x@y.z, registered u2]
--- pure test interpreter (shared state) ---
user=User[id=u3, email=t@est.io]
emails=[t@est.io|Welcome]
logs=[register t@est.io, registered u3]
db=[t@est.io]
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Tagless Final.

Related Patterns

- Contrast Tagless Final with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Tagless Final → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.